



Actividad 2: Clases y Objetos

Programación Orientada a Objetos

Ejercicios de programación orientada a objetos con Java y UML
Leonardo Bermón Angarita

Facultad de Ingeniería

Universidad Nacional de Colombia
Docente: Walter Hugo Arboleda Mazo

Ximena Muñoz Lara
Octubre de 2026

Contenido

Ejercicio 2.1 – Persona.....	3
Diagrama de Clase:.....	4
URL en GitHub:.....	4
Ejercicio 2.2 – Planeta.....	5
Diagrama de Clase:.....	6
URL en GitHub:.....	6
Ejercicio 2.3 – Automóvil.....	7
Diagrama de Clase:.....	11
URL en GitHub:.....	11
Ejercicio 2.4 – Figuras geométricas.....	12
Diagrama de Clase:.....	15
URL en GitHub:.....	15
Ejercicio 2.5 – Cuenta bancaria.....	17
Diagrama de Clase:.....	18
URL en GitHub:.....	18

Ejercicio 2.1 – Persona

Enunciado: Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellido, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus atributos y un método que imprima en pantalla los valores de los atributos del objeto. En un método main se deben crear dos personas y mostrar los valores de sus atributos en pantalla.

```
class Persona:
    def __init__(self, nombre, apellidos, documento, anio_nacimiento):
        self.nombre = nombre
        self.apellidos = apellidos
        self.documento = documento
        self.anio_nacimiento = anio_nacimiento

    def imprimir(self):
        print("Nombre =", self.nombre)
        print("Apellidos =", self.apellidos)
        print("Número de documento de identidad =", self.documento)
        print("Año de nacimiento =", self.anio_nacimiento)
        print()

def main():
    p1 = Persona("Pedro", "Pérez", "1053121010", 1998)
    p2 = Persona("Luis", "León", "1053223344", 2001)

    p1.imprimir()
    p2.imprimir()

if __name__ == "__main__":
    main()
```

Diagrama de Clase:

Persona
- nombre: str - apellidos: str - documento: str - anio_nacimiento: int
+ __init__(nombre, apellidos, documento, anio_nacimiento) + imprimir(): None

URL en GitHub:

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/persona.py>

Ejercicio 2.2 – Planeta

Enunciado: Se requiere un programa que modele el concepto de un planeta del sistema solar, con los atributos: nombre, cantidad de satélites, masa (kg), volumen (km³), diámetro (km), distancia media al Sol, tipo de planeta de acuerdo con su tamaño (enumerado: GASEOSO, TERRESTRE y ENANO) y si es observable a simple vista. La clase debe tener un constructor, un método que imprima los atributos, un método que calcule la densidad (masa / volumen) y un método que determine si el planeta es exterior (situado más allá del cinturón de asteroides, que se encuentra entre 2.1 y 3.4 UA; 1 UA = 149 597 870 km). En el main se deben crear dos planetas y mostrar sus atributos, su densidad y si es exterior.

```
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = 1
    TERRESTRE = 2
    ENANO = 3

class Planeta:
    def __init__(self, nombre, cantidad_satelites, masa, volumen, diametro,
                 distancia_sol, tipo, es_observable):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa # en kilogramos
        self.volumen = volumen # en kilometros cubicos
        self.diametro = diametro # en kilometros
        self.distancia_sol = distancia_sol # en kilometros
        self.tipo = tipo
        self.es_observable = es_observable

    def imprimir(self):
        print("Nombre del planeta =", self.nombre)
        print("Cantidad de satélites =", self.cantidad_satelites)
        print("Masa del planeta =", self.masa)
        print("Volumen del planeta =", self.volumen)
        print("Diámetro del planeta =", self.diametro)
        print("Distancia al sol =", self.distancia_sol)
        print("Tipo de planeta =", self.tipo.name)
        print("Es observable =", self.es_observable)

    def calcular_densidad(self):
        return self.masa / self.volumen

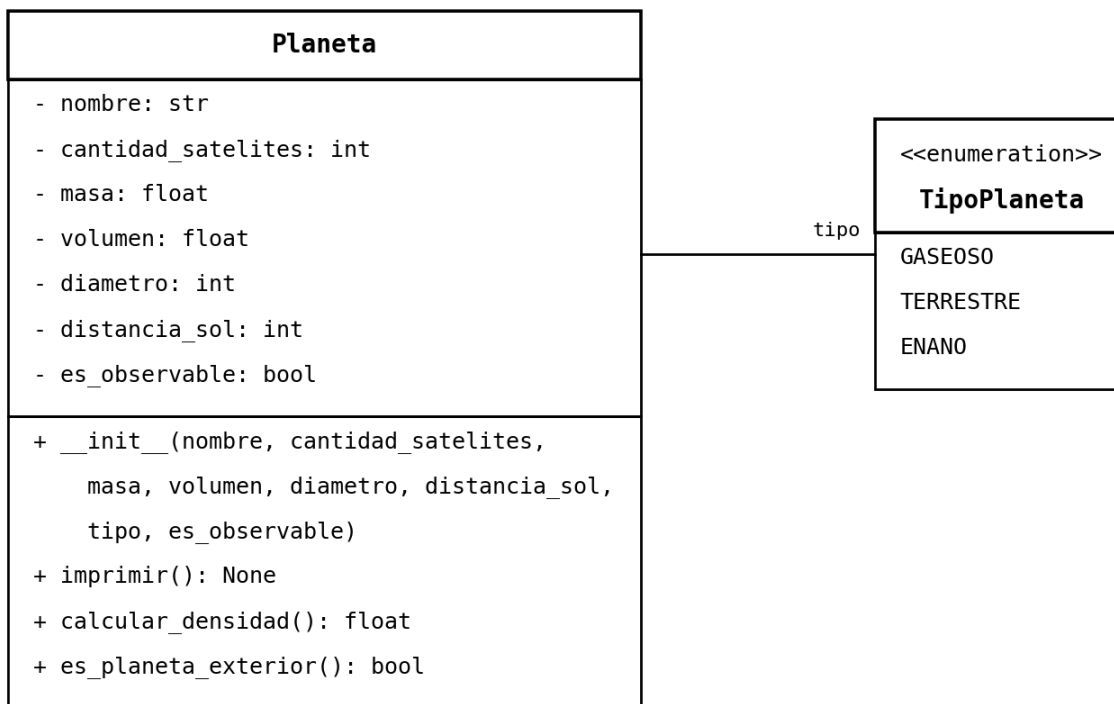
    def es_planeta_exterior(self):
        # el cinturón de asteroides llega hasta 3.4 UA (1 UA = 149597870 km)
        limite = 149597870 * 3.4
        return self.distancia_sol > limite
```

```
def main():
    p1 = Planeta("Tierra", 1, 5.9736E24, 1.08321E12, 12742, 150000000,
                TipoPlaneta.TERRESTRE, True)
    p1.imprimir()
    print("Densidad del planeta =", p1.calcular_densidad())
    print("Es planeta exterior =", p1.es_planeta_exterior())
    print()

    p2 = Planeta("Júpiter", 79, 1.899E27, 1.4313E15, 139820, 750000000,
                TipoPlaneta.GASEOSO, True)
    p2.imprimir()
    print("Densidad del planeta =", p2.calcular_densidad())
    print("Es planeta exterior =", p2.es_planeta_exterior())

if __name__ == "__main__":
    main()
```

Diagrama de Clase:



URL en GitHub:

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/planeta.py>

Ejercicio 2.3 – Automóvil

Enunciado: Se requiere un programa que modele el concepto de un automóvil con los atributos: marca, modelo, motor (litros), tipo de combustible (enumerado), tipo de automóvil (enumerado), número de puertas, cantidad de asientos, velocidad máxima, color (enumerado) y velocidad actual. La clase debe incluir un constructor, métodos get y set, métodos para acelerar, desacelerar y frenar (sin superar la velocidad máxima ni quedar en velocidad negativa, mostrando los mensajes correspondientes), un método para calcular el tiempo estimado de llegada dada una distancia en km, y un método para mostrar los atributos. En el main se crea un automóvil, se pone su velocidad en 100 km/h, se aumenta en 20, se decrementa en 50 y luego se frena, mostrando la velocidad actual en cada cambio.

```
from enum import Enum

class TipoCombustible(Enum):
    GASOLINA = 1
    BIOETANOL = 2
    DIESEL = 3
    BIODIESEL = 4
    GAS_NATURAL = 5

class TipoAutomovil(Enum):
    CIUDAD = 1
    SUBCOMPACTO = 2
    COMPACTO = 3
    FAMILIAR = 4
    EJECUTIVO = 5
    SUV = 6

class Color(Enum):
    BLANCO = 1
    NEGRO = 2
    ROJO = 3
    NARANJA = 4
    AMARILLO = 5
    VERDE = 6
    AZUL = 7
    VIOLETA = 8

class Automovil:
    def __init__(self, marca, modelo, motor, tipo_combustible, tipo_automovil,
                 numero_puertas, cantidad_asientos, velocidad_maxima, color):
        self.marca = marca
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
```

```
self.numero_puertas = numero_puertas
self.cantidad_asientos = cantidad_asientos
self.velocidad_maxima = velocidad_maxima
self.color = color
self.velocidad_actual = 0

# Metodos get
def get_marca(self):
    return self.marca

def get_modelo(self):
    return self.modelo

def get_motor(self):
    return self.motor

def get_tipo_combustible(self):
    return self.tipo_combustible

def get_tipo_automovil(self):
    return self.tipo_automovil

def get_numero_puertas(self):
    return self.numero_puertas

def get_cantidad_asientos(self):
    return self.cantidad_asientos

def get_velocidad_maxima(self):
    return self.velocidad_maxima

def get_color(self):
    return self.color

def get_velocidad_actual(self):
    return self.velocidad_actual

# Metodos set
def set_marca(self, marca):
    self.marca = marca

def set_modelo(self, modelo):
    self.modelo = modelo

def set_motor(self, motor):
    self.motor = motor

def set_tipo_combustible(self, tipo_combustible):
    self.tipo_combustible = tipo_combustible
```

```
def set_tipo_automovil(self, tipo_automovil):
    self.tipo_automovil = tipo_automovil

def set_numero_puertas(self, numero_puertas):
    self.numero_puertas = numero_puertas

def set_cantidad_asientos(self, cantidad_asientos):
    self.cantidad_asientos = cantidad_asientos

def set_velocidad_maxima(self, velocidad_maxima):
    self.velocidad_maxima = velocidad_maxima

def set_color(self, color):
    self.color = color

def set_velocidad_actual(self, velocidad_actual):
    self.velocidad_actual = velocidad_actual

def acelerar(self, incremento):
    # no se puede pasar de la velocidad maxima
    if self.velocidad_actual + incremento <= self.velocidad_maxima:
        self.velocidad_actual = self.velocidad_actual + incremento
    else:
        print("No se puede incrementar a una velocidad superior a la máxima del
automóvil.")

def desacelerar(self, decremento):
    # tampoco puede quedar en velocidad negativa
    if self.velocidad_actual - decremento >= 0:
        self.velocidad_actual = self.velocidad_actual - decremento
    else:
        print("No se puede decrementar a una velocidad negativa.")

def frenar(self):
    self.velocidad_actual = 0

def calcular_tiempo_llegada(self, distancia):
    if self.velocidad_actual == 0:
        print("El automóvil está detenido, no se puede calcular el tiempo.")
        return 0
    return distancia / self.velocidad_actual

def imprimir(self):
    print("Marca =", self.marca)
    print("Modelo =", self.modelo)
    print("Motor =", self.motor)
    print("Tipo de combustible =", self.tipo_combustible.name)
    print("Tipo de automóvil =", self.tipo_automovil.name)
    print("Número de puertas =", self.numero_puertas)
    print("Cantidad de asientos =", self.cantidad_asientos)
```

```
        print("Velocidad máxima =", self.velocidad_maxima)
        print("Color =", self.color.name)

def main():
    auto1 = Automovil("Ford", 2018, 3, TipoCombustible.DIESEL,
                      TipoAutomovil.EJECUTIVO, 5, 6, 250, Color.NEGRO)
    auto1.imprimir()

    auto1.set_velocidad_actual(100)
    print("Velocidad actual =", auto1.get_velocidad_actual())

    auto1.acelerar(20)
    print("Velocidad actual =", auto1.get_velocidad_actual())

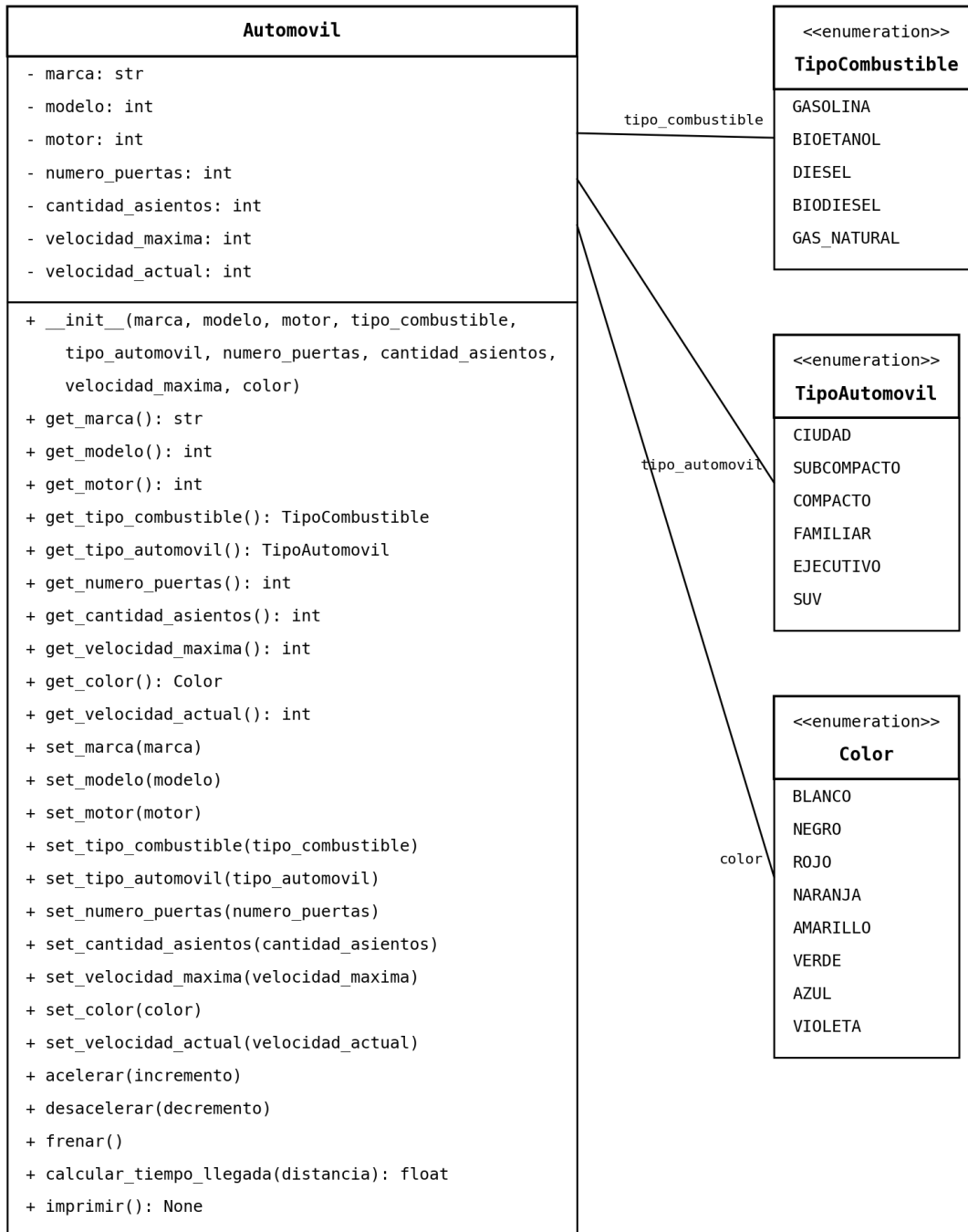
    auto1.desacelerar(50)
    print("Velocidad actual =", auto1.get_velocidad_actual())
    print("Tiempo estimado para 140 km =", auto1.calcular_tiempo_llegada(140), "horas")

    auto1.frenar()
    print("Velocidad actual =", auto1.get_velocidad_actual())

    auto1.desacelerar(20)

if __name__ == "__main__":
    main()
```

Diagrama de Clase:



URL en GitHub:

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/automovil.py>

Ejercicio 2.4 – Figuras geométricas

Enunciado: Se requiere un programa que modele varias figuras geométricas: el círculo (radio), el rectángulo (base y altura), el cuadrado (lado) y el triángulo rectángulo (base y altura). Se requieren métodos para determinar el área y el perímetro de cada figura. Para el triángulo rectángulo además se requiere un método que calcule la hipotenusa y otro que determine qué tipo de triángulo es (equilátero, isósceles o escaleno). Se debe desarrollar una clase de prueba con un método main para crear las cuatro figuras y probar los métodos respectivos.

Archivo 1:

```
import math

class Circulo:
    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return math.pi * self.radio ** 2

    def calcular_perimetro(self):
        return 2 * math.pi * self.radio
```

Archivo 2:

```
class Rectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

    def calcular_perimetro(self):
        return 2 * self.base + 2 * self.altura
```

Archivo 3:

```
class Cuadrado:
    def __init__(self, lado):
        self.lado = lado

    def calcular_area(self):
        return self.lado * self.lado

    def calcular_perimetro(self):
        return 4 * self.lado
```

Archivo 4:

```
import math

class TrianguloRectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura / 2

    def calcular_hipotenusa(self):
        # teorema de pitagoras
        return math.sqrt(self.base ** 2 + self.altura ** 2)

    def calcular_perimetro(self):
        return self.base + self.altura + self.calcular_hipotenusa()

    def determinar_tipo_triangulo(self):
        hipotenusa = self.calcular_hipotenusa()

        if self.base == self.altura == hipotenusa:
            print("Es un triángulo equilátero")
        elif self.base != self.altura and self.base != hipotenusa and self.altura !=
hipotenusa:
            print("Es un triángulo escaleno")
        else:
            print("Es un triángulo isósceles")
```

Archivo 5:

```
from circulo import Circulo
from rectangulo import Rectangulo
from cuadrado import Cuadrado
from triangulo_rectangulo import TrianguloRectangulo

def main():
    figura1 = Circulo(2)
    figura2 = Rectangulo(1, 2)
    figura3 = Cuadrado(3)
    figura4 = TrianguloRectangulo(3, 5)

    print("El área del círculo es =", figura1.calcular_area())
    print("El perímetro del círculo es =", figura1.calcular_perimetro())
    print()

    print("El área del rectángulo es =", figura2.calcular_area())
    print("El perímetro del rectángulo es =", figura2.calcular_perimetro())
```

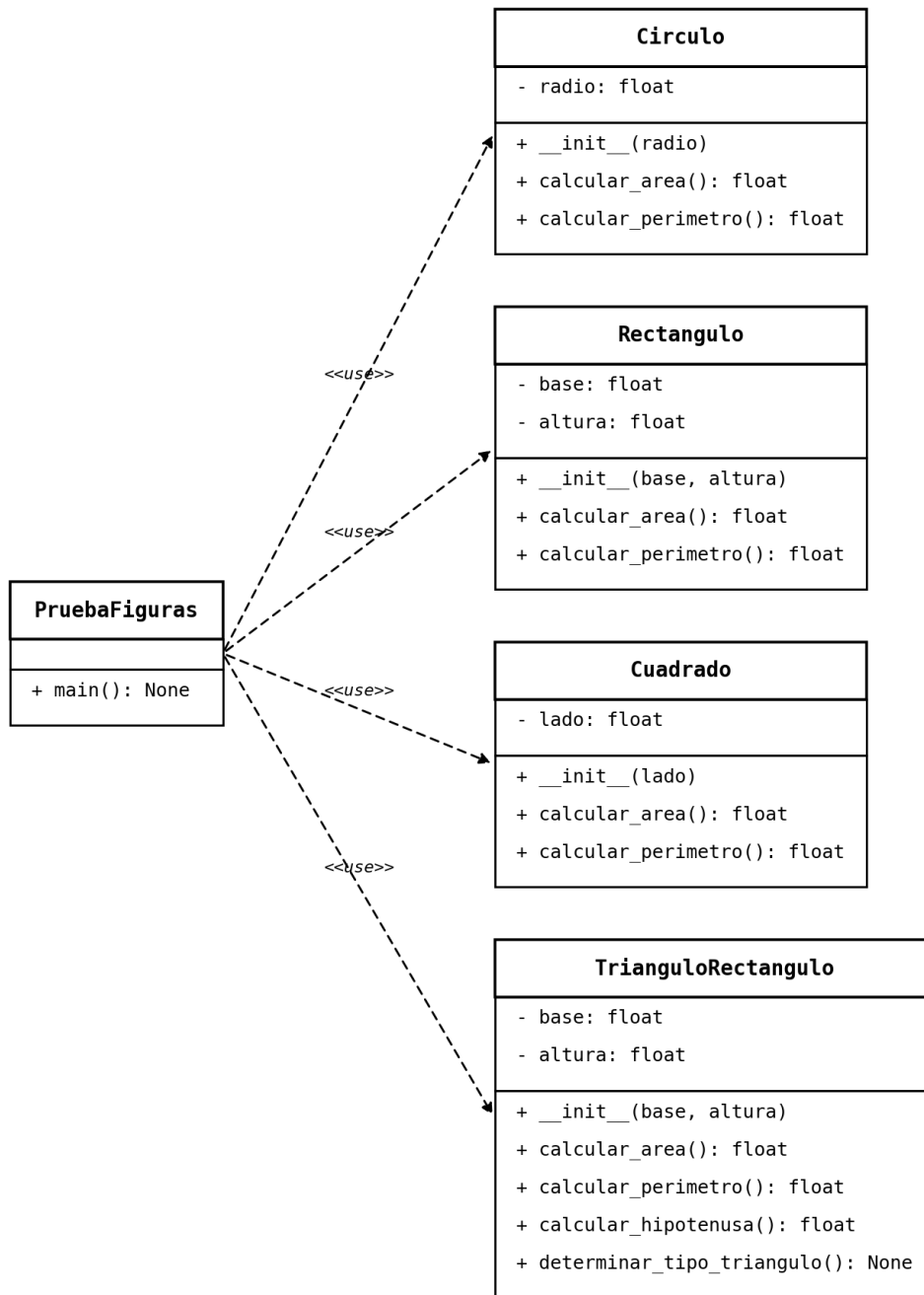
```
print()

print("El área del cuadrado es =", figura3.calcular_area())
print("El perímetro del cuadrado es =", figura3.calcular_perimetro())
print()

print("El área del triángulo es =", figura4.calcular_area())
print("El perímetro del triángulo es =", figura4.calcular_perimetro())
print("La hipotenusa es =", figura4.calcular_hipotenusa())
figura4.determinar_tipo_triangulo()

if __name__ == "__main__":
    main()
```

Diagrama de Clase:



URL en GitHub:

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/circulo.py>

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/rectangulo.py>

<https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/cuadrado.py>

https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/triangulo_rectangulo.py

https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/prueba_figuras.py

Ejercicio 2.5 – Cuenta bancaria

Enunciado: Se requiere un programa que modele una cuenta bancaria con los atributos: nombres del titular, apellidos del titular, número de la cuenta, tipo de cuenta (ahorros o corriente) y saldo. El constructor inicializa los atributos y el saldo inicial es cero. En una cuenta se puede imprimir los valores de sus atributos, consultar el saldo, consignar un valor y retirar un valor (el retiro no se puede realizar si el valor solicitado supera el saldo actual).

```
from enum import Enum

class TipoCuenta(Enum):
    AHORROS = 1
    CORRIENTE = 2

class CuentaBancaria:
    def __init__(self, nombres_titular, apellidos_titular, numero_cuenta, tipo_cuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        # el saldo siempre empieza en cero
        self.saldo = 0

    def imprimir(self):
        print("Nombres del titular =", self.nombres_titular)
        print("Apellidos del titular =", self.apellidos_titular)
        print("Número de cuenta =", self.numero_cuenta)
        print("Tipo de cuenta =", self.tipo_cuenta.name)
        print("Saldo =", self.saldo)

    def consultar_saldo(self):
        print("El saldo actual es =", self.saldo)

    def consignar(self, valor):
        if valor > 0:
            self.saldo = self.saldo + valor
            print(f"Se ha consignado ${valor} en la cuenta. El nuevo saldo es $ {self.saldo}")
            return True
        else:
            print("El valor a consignar debe ser mayor que cero.")
            return False

    def retirar(self, valor):
        # no se puede retirar mas de lo que hay en la cuenta
        if valor > 0 and valor <= self.saldo:
            self.saldo = self.saldo - valor
            print(f"Se ha retirado ${valor} de la cuenta. El nuevo saldo es $
```

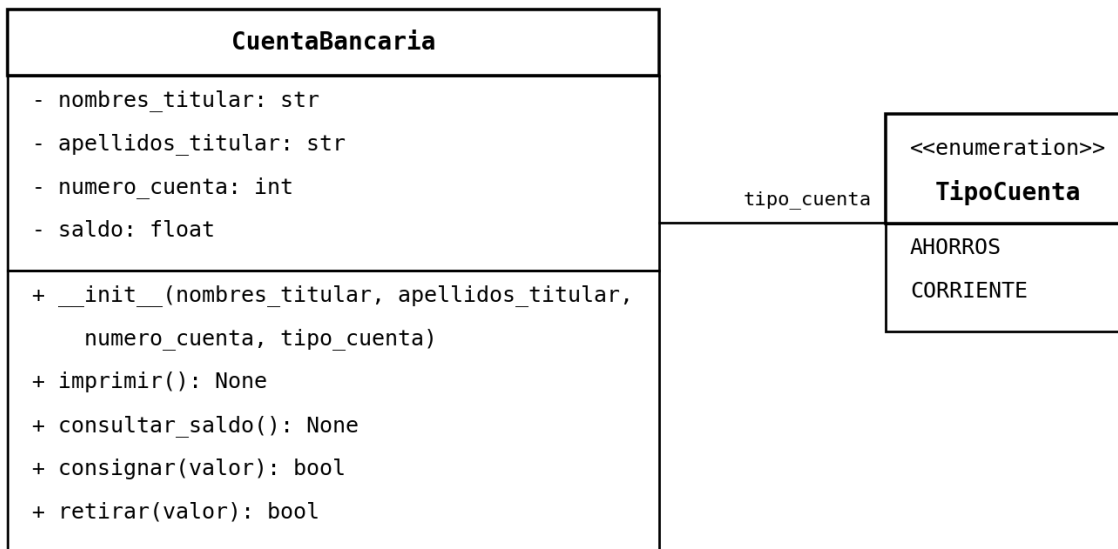
```
{self.saldo}")
    return True
else:
    print("El valor a retirar debe ser mayor que cero y no puede superar el
saldo actual.")
    return False

def main():
    cuenta = CuentaBancaria("Pedro", "Pérez", 123456789, TipoCuenta.AHORROS)
    cuenta.imprimir()

    cuenta.consignar(200000)
    cuenta.consignar(300000)
    cuenta.retirar(400000)
    cuenta.retirar(900000)
    cuenta.consultar_saldo()

if __name__ == "__main__":
    main()
```

Diagrama de Clase:



URL en GitHub:

https://github.com/xmunozl-collab/Actividad-2-POO-2026-2/blob/main/cuenta_bancaria.py